

EECS 348 Assignment 2

GenAI Code Comparison – CEO Email Priority Queue

Author: Prabesh Sapkota

Date: September 17, 2026

1. Objective

The goal of this assignment is to implement a priority queue for a CEO's email inbox using a list-based MaxHeap. Emails are prioritized first by sender category and then by date. The sender priority is Boss, Subordinate, Peer, ImportantPerson, and OtherPerson. If two emails have the same sender category, the newer email must be read first. The program supports EMAIL, NEXT, READ, and COUNT commands and handles empty queues and repeated commands.

2. GenAI Programs

Copilot Program 1: Copilot. I accessed through the Copilot application.

GenAI Program 2: Google Gemini. I accessed Google Gemini through the Gemini website.

Prompt used for both:

Write a C program that implements a list-based MaxHeap to create a priority queue for a CEO's email inbox. The input consists of EMAIL <sender category>, <subject line>, <date>, NEXT, READ, and COUNT commands. Sender categories have the following priority: Boss, Subordinate, Peer, ImportantPerson, and OtherPerson. If two emails have the same sender category, the newest email should have higher priority. EMAIL subjects may contain spaces but not commas, and dates are in MM-DD-YYYY format. NEXT should display the highest-priority email without removing it, READ should remove the highest-priority email without displaying it, and COUNT should display the number of unread emails. The program should use a list-based MaxHeap implemented from scratch, handle empty queues and repeated commands correctly, and read commands from an input file.

3. Generated Code Analysis

Copilot: The first program uses a dynamically allocated array. It defines Email and MaxHeap structures and separate functions for category priority, date conversion, email comparison, heap insertion, heap deletion, heapify-up, heapify-down, and input processing. It uses dynamic resizing when the heap becomes full.

Gemini: The second program uses a fixed array with MAX_EMAILS set to 500. It defines Email and MaxHeap structures and functions including getCategoryPriority, convertDate, compareEmails, heapifyUp, and heapifyDown. The Gemini code supplied for comparison was incomplete because the pasted response stopped during heapifyDown, so I did not invent the missing code.

4. Correctness

Both programs use the correct general MaxHeap approach. They compare sender category first and date second. The Copilot implementation correctly separates NEXT, which only examines the root, from READ, which removes the root. It also safely handles an empty heap. I improved the final version by adding an insertion sequence so that emails with identical category and date have deterministic ordering.

The Gemini implementation also has the correct comparison structure in the supplied portion, but its fixed 500-email limit is less flexible. Because the supplied Gemini code is incomplete, its complete execution cannot be verified from the provided response.

5. Execution Time

For both implementations, EMAIL insertion is $O(\log n)$ for the heap operation, READ is $O(\log n)$, NEXT is $O(1)$, and COUNT is $O(1)$. Copilot dynamic resizing can require $O(n)$ during an individual resize, but capacity is doubled, giving amortized efficient insertion. The MaxHeap avoids scanning all emails to find the highest-priority email.

6. Space Complexity

Both approaches use $O(n)$ space for n unread emails. The Copilot implementation dynamically allocates only the storage needed by the current heap and grows it when necessary. Gemini reserves a fixed array capable of holding 500 emails, which gives predictable storage but imposes a fixed maximum.

7. Maintainability

The Copilot program is organized into small functions with clear responsibilities, including heap creation, insertion, removal, comparison, parsing, and output. Its dynamic capacity also avoids changing a hard-coded maximum when requirements change. The Gemini version is straightforward, but the fixed-size array makes it less flexible.

8. Selected Program

I selected the Copilot program as the basis for my final implementation. It already had a complete list-based MaxHeap, dynamic memory management, separate heap operations, file processing, and the required commands. Its organization also made it easier to modify and test.

9. Improvements Made

Correctness: Added deterministic tie handling using an insertion sequence. Added safer parsing and explicit handling for empty NEXT and READ operations.

Execution time: Kept the MaxHeap because it provides $O(1)$ NEXT and $O(\log n)$ insertion and READ operations instead of requiring a linear scan.

Space complexity: Kept dynamic array storage, maintaining $O(n)$ space while avoiding an arbitrary fixed maximum.

Maintainability: Separated responsibilities into functions such as senderPriority, dateToValue, higherPriority, ensureCapacity, heapifyUp, heapifyDown, insertHeap, removeMax, peekMax, parseEmailLine, and printEmail. Added comments explaining the code.

10. Testing and Debugging

I tested the final program using the assignment's sample input and verified that it produces the required sequence: 6 emails initially, the newer Boss email first, the older Boss email second after READ, and 4 emails remaining after two READ commands. I also tested NEXT and READ on an empty queue, repeated NEXT commands, repeated READ commands, different sender categories, and equal category/date cases.

11. Conclusion

Both GenAI programs used a MaxHeap and therefore have efficient priority-queue operations. I used the Copilot implementation as the starting point and improved correctness, input handling, memory management, deterministic ordering, organization, and maintainability. The resulting program was compiled and debugged as required.

12. Final Source Code

```
/*
 * EECS 348 Assignment 2
 * Program: CEO Email Priority Queue
 *
 * Description:
 * This C program uses a list-based MaxHeap to prioritize emails for a CEO.
 * Emails are ordered by sender category first and newest date second.
 *
 * Inputs:
 * A test file containing EMAIL records followed by NEXT, READ, and COUNT commands.
 *
 * Outputs:
 * The program displays the requested email information and the current unread count.
 *
 * Collaborators:
 * None.
 *
 * Other sources for code:
 * Copilot was used to generate and explain an initial implementation.
 * The author reviewed, modified, and tested the implementation.
 *
 * Author:
 * Prabesh Sapkota
 *
 * Creation date:
 * September 17, 2026
 *
 * Revision date:
 * September 17, 2026
 *
 * Revisions:
 * Implemented a list-based MaxHeap, added date/category comparison,
 * handled repeated NEXT and READ commands, and added comments.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_SENDER 32
#define MAX_SUBJECT 256
#define MAX_LINE 512

typedef struct {
    char sender[MAX_SENDER];
    char subject[MAX_SUBJECT];
    char date[11];
    int dateValue;
    long sequence;
} Email;

typedef struct {
    Email *items;
    int size;
    int capacity;
} MaxHeap;

int senderPriority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4; if
    (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2; return
    1;
}

int dateToValue(const char *date) { int
    month = 0;
    int day = 0;
    int year = 0;
    sscanf(date, "%2d-%2d-%4d", &month, &day, &year);
    return year * 10000 + month * 100 + day;
}

int higherPriority(const Email *a, const Email *b) { int
    priorityA = senderPriority(a->sender);
    int priorityB = senderPriority(b->sender);

    if (priorityA != priorityB) return
        priorityA > priorityB;

    if (a->dateValue != b->dateValue) return
        a->dateValue > b->dateValue;

    return a->sequence < b->sequence;
}

void swapEmails(Email *a, Email *b)
{ Email temp = *a;
```

```
    *a = *b;
    *b = temp;
}

void initHeap(MaxHeap *heap) {
```

```

    heap->items = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

void destroyHeap(MaxHeap *heap)
{
    free(heap->items);
    heap->items = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

void ensureCapacity(MaxHeap *heap) {
    if (heap->size < heap->capacity) return;

    int newCapacity = (heap->capacity == 0) ? 8 : heap->capacity * 2; Email
    *newItems = realloc(heap->items, newCapacity * sizeof(Email));

    if (newItems == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        destroyHeap(heap);
        exit(EXIT_FAILURE);
    }

    heap->items = newItems;
    heap->capacity = newCapacity;
}

void heapifyUp(MaxHeap *heap, int index) { while
(index > 0) {
    int parent = (index - 1) / 2;

    if (!higherPriority(&heap->items[index],
                      &heap->items[parent]))
        break;

    swapEmails(&heap->items[index], &heap->items[parent]);
    index = parent;
}
}

void heapifyDown(MaxHeap *heap, int index)
{ while (1) {
    int left = 2 * index + 1;
    int right = 2 * index + 2;
    int largest = index;

    if (left < heap->size &&
        higherPriority(&heap->items[left],
                      &heap->items[largest]))
        largest = left;

    if (right < heap->size &&
        higherPriority(&heap->items[right],
                      &heap->items[largest]))
        largest = right;

    if (largest == index)
        break;

    swapEmails(&heap->items[index], &heap->items[largest]); index
    = largest;
}
}

void insertHeap(MaxHeap *heap, Email email)
{
    ensureCapacity(heap);
    heap->items[heap->size] = email;
    heap->size++;
    heapifyUp(heap, heap->size - 1);
}

void removeMax(MaxHeap *heap) { if
(heap->size == 0) return;

    heap->items[0] = heap->items[heap->size - 1];
    heap->size--;

    if (heap->size > 0)
        heapifyDown(heap, 0);
}

const Email *peekMax(const MaxHeap *heap)
{
    if (heap->size == 0) return NULL;
    return &heap->items[0];
}

void printEmail(const Email *email)
{
    printf("Next email:\n");
    printf("Sender: %s\n", email->sender);
    printf("Subject: %s\n", email->subject);
    printf("Date: %s\n", email->date);
}

```

```
}

int parseEmailLine(const char *line, Email *email, long sequence)
{ char date[11];

  if (sscanf(line, "EMAIL %31[^,],%255[^,],%10s",
```

```

        email->sender, email->subject, date) != 3) return
0;

strcpy(email->date, date, sizeof(email->date) - 1);
email->date[sizeof(email->date) - 1] = '\0';
email->dateValue = dateToValue(email->date);
email->sequence = sequence;

return 1;
}

int main(int argc, char *argv[]) { if
(argc != 2) {
    fprintf(stderr, "Usage: %s <test_file>\n", argv[0]);
    return EXIT_FAILURE;
}
FILE *file = fopen(argv[1], "r");

if (file == NULL) {

    fprintf(stderr, "Unable to open test file: %s\n", argv[1]); return
EXIT_FAILURE;
}

MaxHeap heap;
initHeap(&heap);

char line[MAX_LINE];
long sequence = 0;

while (fgets(line, sizeof(line), file) != NULL)
{ line[strcspn(line, "\r\n")] = '\0';

    if (strncmp(line, "EMAIL ", 6) == 0)
    { Email email;

        if (parseEmailLine(line, &email, sequence))
        { insertHeap(&heap, email);
            sequence++;
        }
    }
    else if (strcmp(line, "NEXT") == 0) { const
        Email *email = peekMax(&heap);

        if (email != NULL)
            printEmail(email);
    }
    else if (strcmp(line, "READ") == 0)
    { removeMax(&heap);
    }
    else if (strcmp(line, "COUNT") == 0) {
        printf("There are %d emails to read.\n", heap.size);
    }
}

destroyHeap(&heap);
fclose(file);

return 0;
}

```